UNIVERSIDAD CATOLICA DEL MAULE

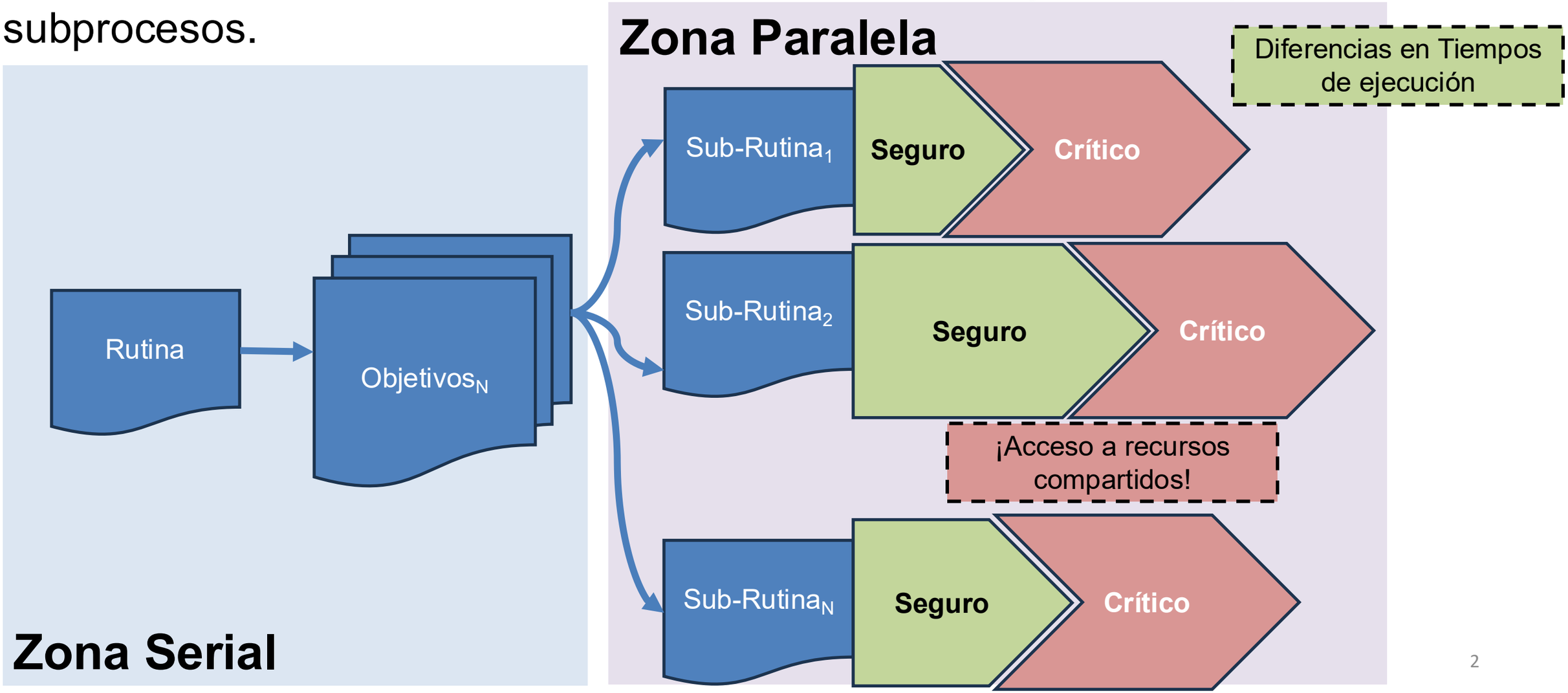
Computación de Alto Rendimiento

Javier E. Rojas Núñez

jerojas@ucm.cl

Multiprocesing

Los procesos paralelizados permiten distribuir el trabajo en multiples subprocesos.



UNIVERSIDAD CATOLICA DEL MAULE

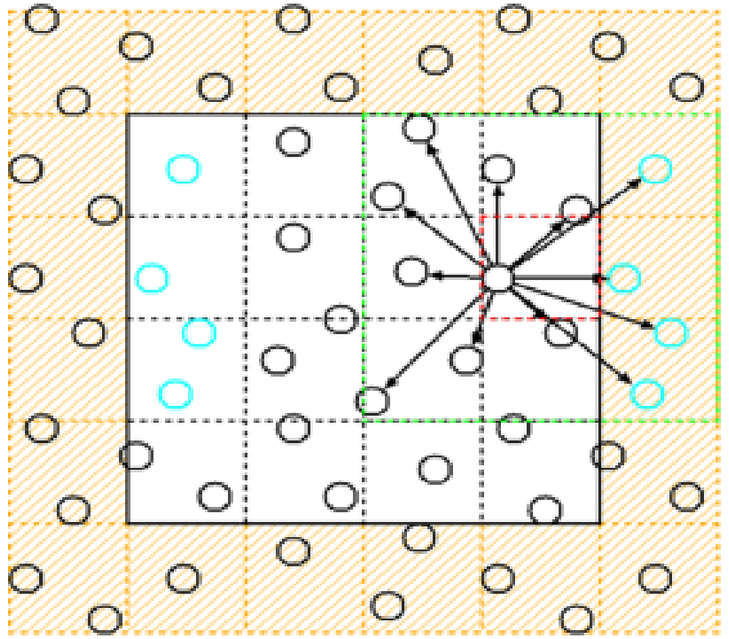


Sincronización

Ejemplo de sincronización

Hay procesos en donde la sincronización puede ser gran utilidad para sincronizar procesos en paralelo. Por ejemplo en Dinámica molecular

- La simulación se "reparte" los elementos a simular por volumen de espacio
- Cada proceso calcula los pasos siguientes para cada uno de los elementos a su cargo
- Terminado esto, se debe esperar a que los procesos vecinos terminen de actualizar sus resultados



¿Por qué es necesario sincronizar?

Para trabajar en equipo se debe trabajar coordinados. Es por esto que la programación paralela requiere de herramientas que permitan coordinar los trabajos individuales creados por el proceso padre.

- Barreras: Punto común de espera
- Cerrojos: Permiso exclusivo de ejecución de un proceso
- Semaforos: Permiso exclusivo de ejecución de mútples procesos



Barreras

Las barrera permiten a todos los hijos elejir un punto común dentro de su rutina en la cual tendrán que esperar hasta que todos lleguen a ese mismo punto.

Esto requiere de crear un objeto de Barrera

```
barrier = mp.Barrier(num_processes)
```

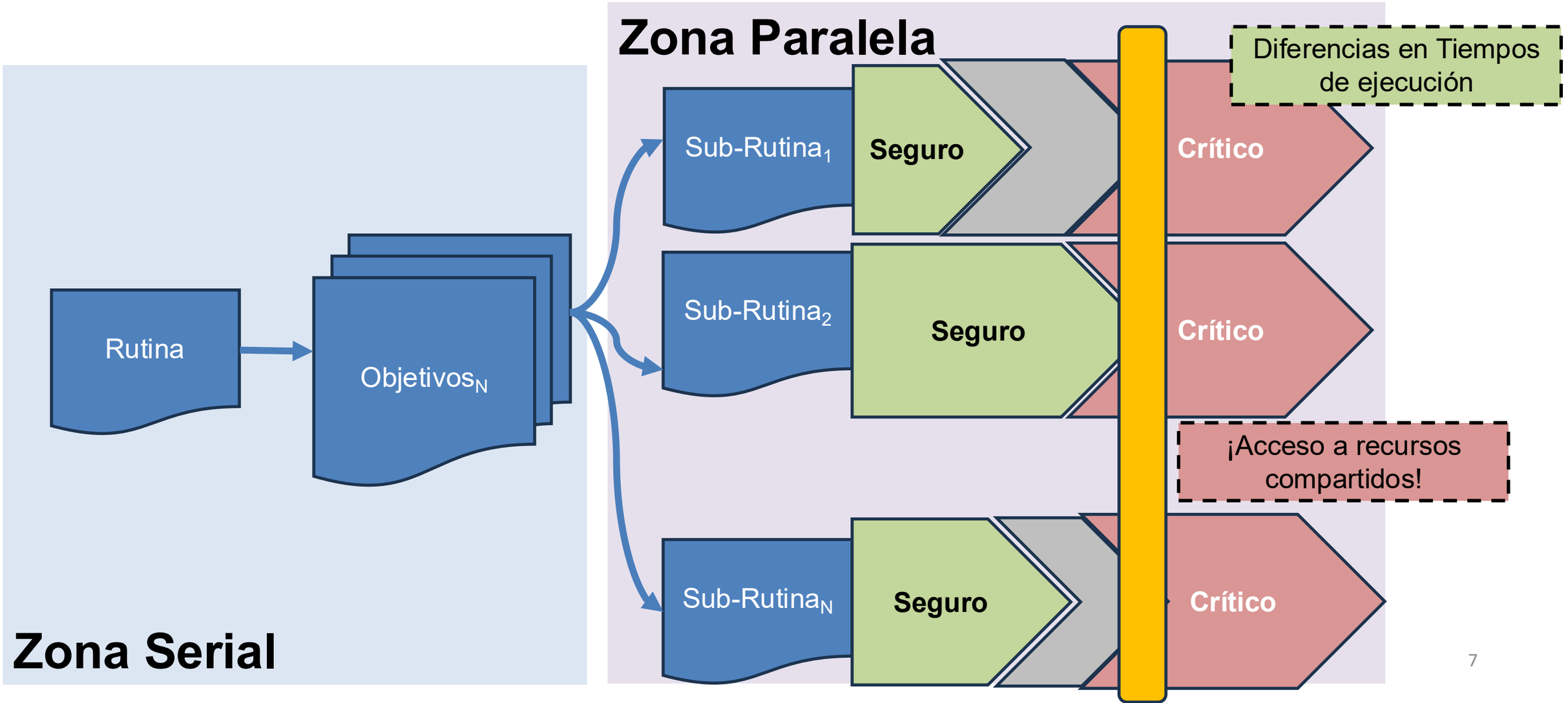
Y luego utilizarlo dentro del código hijo

```
print(f"Proceso {process_id} llegó a la
      barrera.")
barrier.wait()
```



Barreras

La barrera mantiene los procesos en sincronía.



Cerrojos y sincronización

Los cerrojos funcionan como una puerta con llave. Los procesos tienen una única copia que compartir, y solo un numero limitado puede acceder a la vez

Una llave debe crearse

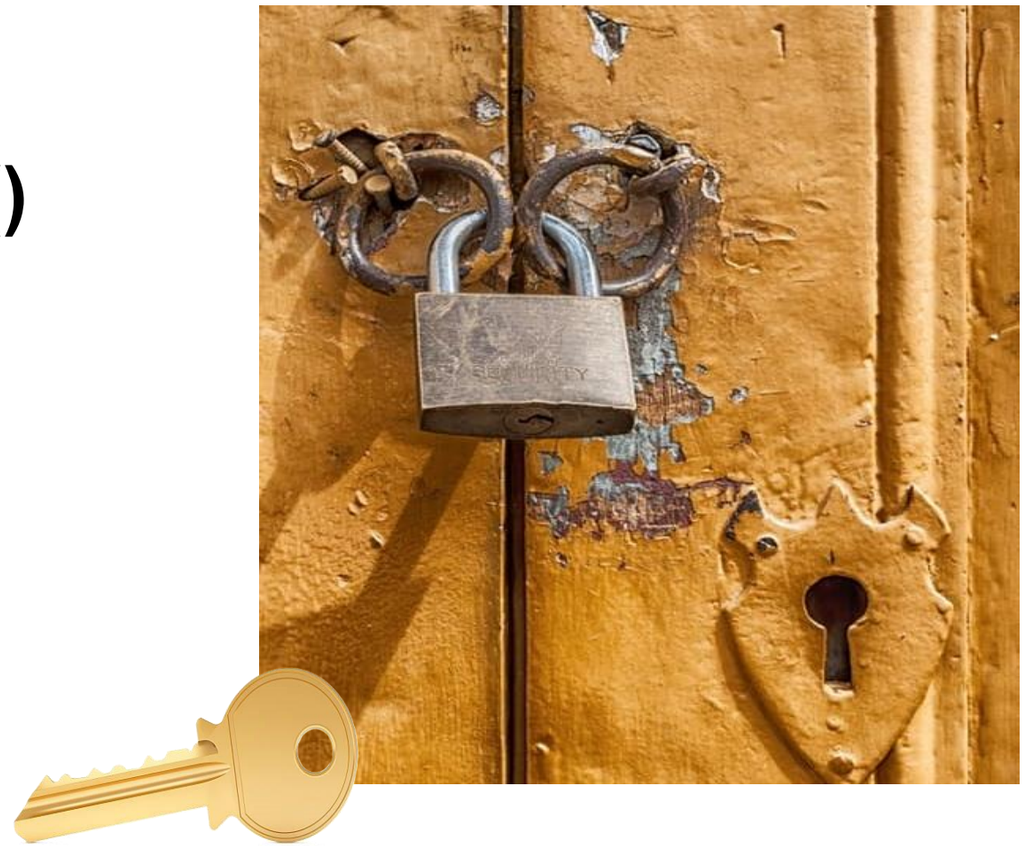
lock = multiprocessing.Lock()

Solo un proceso puede pasar la barrera con su llave

lock.acquire()

…

lock.release()



Cerrojo semaforo

El cerrojo semaforo es parecido al cerrojo simple, solo que aqui se pueden tener multiple llaves validas a la vez



Una llave debe crearse

lock = mp.Semaphore(1)

Solo un proceso puede pasar la barrera con su llave

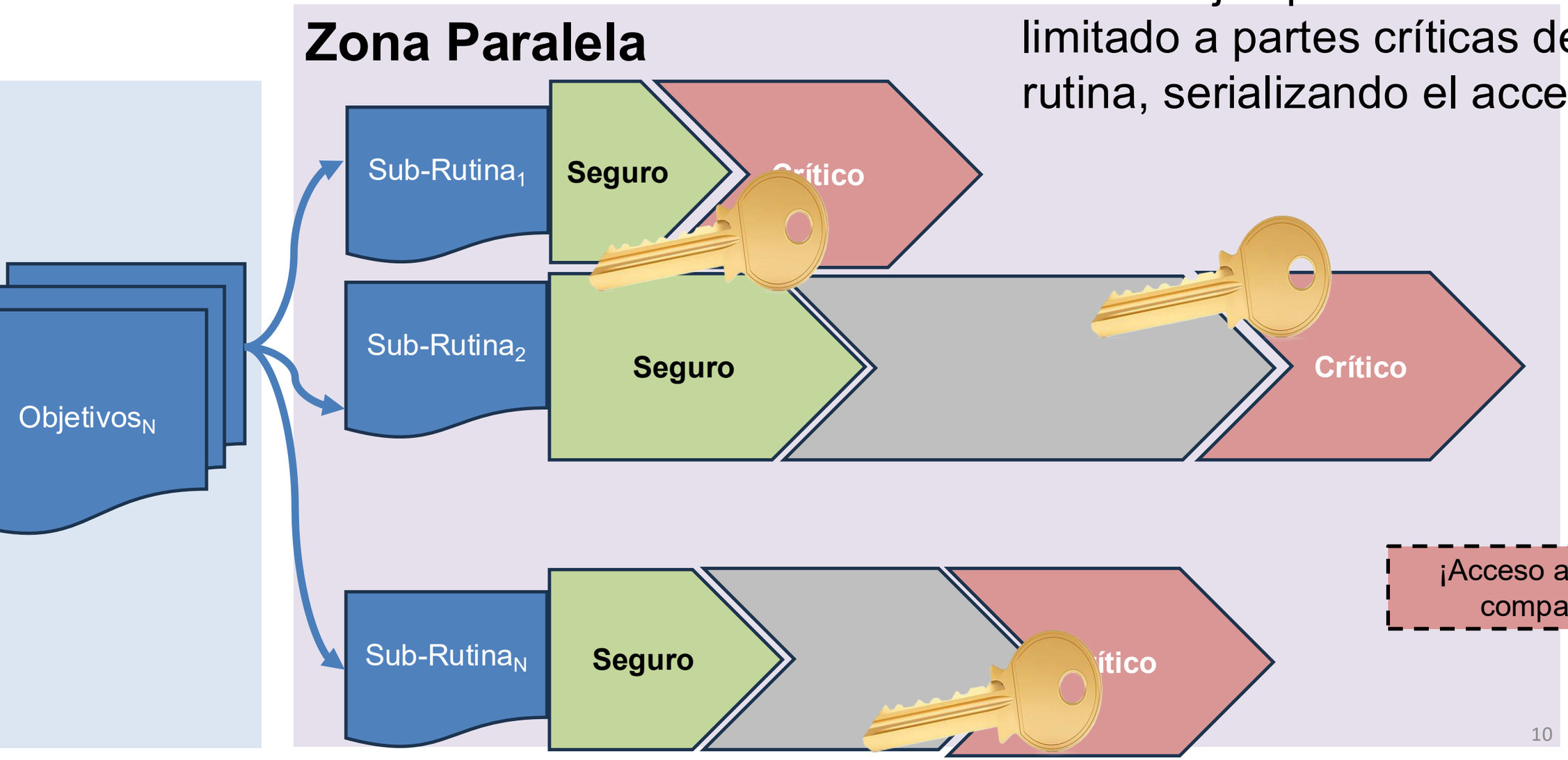
lock.acquire()

...

lock.release()

Cerrojos

Los cerrojos permiten un acceso limitado a partes críticas de la rutina, serializando el acceso.



¡Acceso a recursos compartidos!



Midiendo la eficiencia

UNIVERSIDAD CATOLICA DEL MAULE

¿Cuánto podemos esperar del procesameinto paralelo?

Por razones evidentes, la máxima aspiración de un código paralelo completar la tarea N veces más rápido que el procesos serial.

Esto no es posible, debido a los timepos extra consumidos en la sincronización y espera por comunicación



¿Cómo medimos la eficiencia?

Para medir la eficiencia de uestra paralelización solo debemos considerar el tiempo que tomaría realizarse la tarea en un único proceso serial.
Luego se compara con el tiepo que toma el proceso en paralelo

$$\mu = \frac{\text{tiempo serial}}{n * \text{tiempo paralelo}}$$



Aplicando a un código

Utilizando la librería time encontraremos funciones útiles para medir el tiempo de ejecución. Por ejemplo la herramienta perf_counter()

Esta función porporciona un contador interno que da una razonable prescición temporal, medida en segundos.



$$t = \text{contador final} - \text{contador inicial}$$

UNIVERSIDAD CATOLICA DEL MAULE

Implemente el contador





Actividad

Acelerando un proceso

UNIVERSIDAD CATOLICA DEL MAULE

Mostraremos un algoritmo serial el cual tendrán que utilizar alguna (o varias) de las técnicas vistas en clase para paralelizar y acelerar la finalización de la tarea. Este proceso requiere de organizar en que sector del espacio se encuentra un conjunto de átomos dentro de un espacio cúbico.

- Adaptar la rutina mostrada para trabajar con multiples procesos coordinados. Identifique que parte de la rutina es segura de ejecutar en paralelo y cual requerirá de acceso restringido.
- Permita que los procesos escriban en disco pero evitando que varios procesos accedan al archivo a escribir a la vez.
- Analice la velocidad en la que se ejecuta el algoritmo en serial y paralelo al momento de procesar los datos.
- Implemente un contador dentro del algoritmo para contabilizar el tiempo invertido en la sección restringida y no restringida.

```
def fun_paralela(iid):
  ...

if __name__ == '__main__':
    p_num = 8
    arr = mp.Array('i', [0 for
                         lock=False)
    # Creando lista de proceso
    procs = ???

    # Corriendo tareas
    for p in procs:
        p.start()

    # Esperar a que terminen l
    for p in procs:
        p.join()
```